

Roteiro de Apresentação — Árvore B em Memória Secundária

Duração alvo: ~45 minutos · 37 slides · ~1:00–1:15 por slide · AED-PG-2026

Como usar: cada slide tem a **mensagem-chave** (o que a banca tem que levar) e a **fala** (texto natural, pode adaptar). As frases em *itálico* são transições. No fim há um bloco de **perguntas prováveis**. >

Público: suponha que parte da plateia não conhece Árvore B. Não precisamos **ensinar** a estrutura, mas a linguagem deve ser acessível — fale em termos de "arquivo no disco" e "menos acessos = mais rápido". > **Ritmo:** 37 slides em ~45 min. Os slides marcados com (~) são candidatos a encurtar se o tempo apertar (leia só o título); os com (*) merecem mais tempo.

#	Slide	Alvo
1	Capa	1:00
2	Roteiro (~)	0:45
3	Motivação: memória × disco (*)	1:15
4	Árvore B: parâmetros (anatomia do nó) (*)	1:30
5	Árvores geradas pelo Graphviz	0:45
6	Layout do arquivo no disco (*)	1:30
7	Problema e objetivos	1:30
8	O índice e a métrica mais fiel	1:30
9	Decisões de implementação	1:00
10	A classe DiskManager (1 nó por I/O)	1:30
11	Métodos: visão geral (~)	0:45
12	Busca m-way	1:30
13	Inserção e split	1:30
14	Remoção	1:30
15	Reaproveitamento — free list	1:15
16	Resumo do que o código faz (~)	0:45
17	Ferramentas utilizadas	1:00
18	Demonstração — o programa rodando	1:30
19	Os experimentos (matriz)	1:00
20	Metodologia e ambiente	1:30
21	Métricas utilizadas	1:00
22	Impacto de m: I/O por busca (*)	2:00
23	Impacto de m: altura	1:00
24	Resultados por operação	1:00
25	Escala do conjunto (N)	1:15
26	Comparação: Árvore B × Array ordenado (*)	1:45
27	Reaproveitamento de nós (churn)	1:00
28	Dois sistemas: tempo (wall)	1:15
29	Dois sistemas: CPU e determinismo (*)	1:45

#	Slide	Alvo
30	Dois sistemas: validação cruzada (~)	0:45
31	Dois sistemas: escala	1:00
32	E se usássemos union? (*)	1:45
33	Dificuldades técnicas	1:00
34	Vantagens × desvantagens	1:15
35	Aplicações práticas	0:45
36	Conclusão	1:30
37	Referências / encerramento	0:30

Soma ≈ 45 min. Folga: corte os (~); sobra: detalhe os (*) e abra perguntas no fim de cada parte.

Parte 1 — A estrutura

1. Capa. Mensagem: o que é o trabalho em uma frase. "Nosso trabalho é uma **Árvore B de ordem m que opera estritamente em memória secundária** — a árvore vive num arquivo no disco e toda operação lê ou escreve **um nó por vez**, nunca a árvore inteira." Apresentem-se e citem o foco na avaliação experimental.

2. Roteiro. (~) Mensagem: mapa da fala. "Primeiro a estrutura — motivação, como o nó e o arquivo são, os métodos e uma demo ao vivo. Depois a avaliação — experimentos, impacto de m, uma comparação com um método mais simples, e a discussão final."

Passe rápido.

****3.** Motivação: memória × disco. (*) **Mensagem: disco é o gargalo. "RAM é rápida, mas pequena e volátil; disco é enorme e permanente, porém ~10⁶ vezes mais lento. Num banco de dados real, o verdadeiro gargalo é o número de acessos (leitura/escrita) ao disco. Logo, o segredo da velocidade é fazer o menor número de acessos**."**

"Para isso, vamos ver como é um nó dessa árvore."

****4.** Árvore B: parâmetros (anatomia do nó). (*) **Mensagem: o que há dentro de um nó. "Cada nó é um registro de tamanho fixo, lido de uma vez: o campo n (quantas chaves), o vetor de ponteiros A[] (os RRN dos filhos) e o vetor de chaves K[]. A ordem m define o limite: até m-1 chaves e m filhos por nó. E a regra de ouro: 1 nó = 1 acesso ao disco**."** "Na prática, é assim que esses nós aparecem."

5. Árvores geradas pelo Graphviz. Mensagem: m maior = mais raso. "Estes grafos são exportados **pelo próprio código**. Cada caixa é um nó no disco (o número é o RRN). Com **m=3** a árvore precisa de mais níveis; com **m=4**, mais chaves por nó → ela tende a ser **mais rasa** — e árvore mais rasa significa **menos acessos**." "E como esses nós ficam organizados no arquivo?"

****6.** Layout do arquivo no disco. (*) **Mensagem: o arquivo é um vetor de registros. "O arquivo é um vetor de páginas de tamanho fixo, numeradas por RRN. O RRN 0 é o header — guarda a posição da raiz, o total de nós e a cabeça da free list; com um único acesso já sabemos onde começa a árvore. Qualquer nó é achado por cálculo direto: offset = RRN × PAGE_SIZE. E nós removidos entram numa lista de livres** para serem reusados."** "Com a estrutura na mão, o que o enunciado pedia."

7. Problema e objetivos. Mensagem: o que o enunciado pede. "Implementar uma classe Árvore B de ordem m que **resida em memória secundária** — um arquivo em disco. A restrição central: **nunca carregar a árvore inteira na RAM**, cada operação acessa **um nó por vez**. Operações: busca, inserção e remoção. E o objetivo que assumimos:

avaliar o desempenho variando os parâmetros." "Mas qual a métrica honesta dessa avaliação?"

8. O índice e a métrica mais fiel. Mensagem: índice + métrica honesta. "Esse layout (header + RRNs + páginas fixas) faz a Árvore B funcionar como um **índice**: em vez de ler o arquivo todo, ela segue só o caminho **raiz → folha**, começando do header num único acesso. E a métrica mais fiel **não é o relógio** — é o **número de acessos ao disco**, que independe do hardware." "Vamos às decisões de projeto."

9. Decisões de implementação. Mensagem: as decisões centrais. "Ordem m como **constante de compilação** (M); **raiz no header**; **um nó por I/O**, nunca a árvore completa; e **free list** no próprio arquivo para reaproveitar nós. À direita, uma Árvore B real ($m=3$) exportada pelo código — o número é o RRN no disco." "Toda a E/S passa por uma classe única."

10. A classe DiskManager — um nó por I/O. Mensagem: onde mora o '1 nó por I/O'. "Toda leitura/escrita passa pelo **DiskManager**: quando precisamos analisar um nó, fazemos um **readNode**; se ele muda, um **writeNode** — sempre **um registro de PAGE_SIZE**. E é aqui que vive o **contador de acessos**, a métrica central. A lógica da árvore só enxerga o disco por essa classe, então nada vai inteiro para a RAM."

"Agora os métodos."

11. Métodos: visão geral. (~) Mensagem: as três operações + ferramentas. "Busca m -vias, inserção bottom-up com split, remoção com sucessor/redistribuição/fusão, mais as ferramentas de monitoramento (printTree, height, exportDot). Vou detalhar uma a uma." Não leia item a item.

12. Busca m -way. Mensagem: desce por faixas. "Em cada nó, as chaves dividem o universo em **faixas**; descemos pela faixa certa até a folha. O custo é $\approx$ **a altura** da árvore, em acessos — pouquíssimos." "Inserir é parecido, até o nó estourar."

13. Inserção e split. Mensagem: split mantém o balanço. "Bottom-up: acha a folha e insere em ordem. Se o nó **estoura** (passa de $m-1$ chaves), fazemos **split**: a

mediana sobe ao pai. Isso pode propagar e até criar uma nova raiz — é exatamente o que mantém a árvore **sempre balanceada**." "Remover é o caso mais delicado."

14. Remoção. Mensagem: sucessor, redistribuição, fusão. "Se a chave está num nó interno, trocamos pelo **sucessor in-order**. Uma folha pode ficar em **underflow**; reparamos com **redistribuição** (empresta de um irmão) ou **fusão** (junta com o irmão + a chave do pai). A fusão pode propagar para cima e baixar a árvore." "E o que fazemos com o nó que sobra de uma fusão?"

15. Reaproveitamento — free list. Mensagem: como a free list funciona. "Ao remover, o nó liberado vira **LIVRE** e entra numa pilha encadeada no próprio arquivo (cabeça no header, via freeNode). A próxima alocação (allocNode) **reusa um livre** antes de crescer o arquivo. É **ligável** (setReuse) — usamos isso para medir o ganho de espaço no experimento de churn." "Num slide, tudo que o código faz."

16. Resumo do que o código faz. (~) Mensagem: núcleo + extras. "À esquerda o **núcleo** — Árvore B 100% em disco, ordem parametrizável, CRUD com split/merge, persistência por RRN. À direita os **extras** de avaliação: contador de acessos, free list ligável, printTree/height/exportDot. O núcleo é a estrutura; os extras existem para **medi-la**." "Com que ferramentas montamos isso?"

17. Ferramentas utilizadas. Mensagem: a stack do projeto. "Implementação: **C++17** (g++ -O2), Make, std::fstream binário com registros fixos, Graphviz para os grafos. Avaliação e material: Python + matplotlib, python-pptx/reportlab. Tudo **versionado em Git** e com experimentos **reprodutíveis** (scripts + CSV)." "Chega de slides: vamos ver rodando."

18. Demonstração — o programa rodando. Mensagem: é real. "Captura de uma execução de verdade: make $M=3$, o **menu interativo** com CRUD + métricas, e o grafo exportado pelo próprio programa. Depois com $m=5$, onde a árvore fica mais rasa." "E como medimos isso de forma

sistemática?"

Parte 2 — A avaliação

19. Os experimentos (matriz). Mensagem: a matriz cobre o enunciado. "Quatro experimentos: **impacto de m** ($N=10^5$), **escala de N** (10^3 – 10^6), **ocupação com/sem reuso**, e **CPU vs I/O** — tudo em modos aleatório e sequencial, validado em **duas máquinas**." "Como exatamente medimos."

20. Metodologia e ambiente. Mensagem: rigor. "Um **driver não interativo** emite CSV; cada configuração é **reconstruída do zero**; medimos com **getrusage** (CPU usuário, CPU sistema e espera de I/O). Duas máquinas: **Notebook x Titan (USP)**, com automação." "Quais métricas exatamente."

21. Métricas utilizadas. Mensagem: a honesta é o contador. "A central é **acessos ao disco**. Também: **altura** ($\sim \log_m N$), **ocupação** (nós, livres, bytes) e **tempo** decomposto (wall, CPU usuário/sistema, espera de I/O)." "O resultado central: o impacto de m."

****22. Impacto de m: I/O por busca. (*) Mensagem: despenca e satura. "Com $m=3$, $\sim 13,25$ acessos por busca (altura 14). Subindo m, despenca: $m \geq 512 \rightarrow 2,00$ acessos (altura 2). Mas o ganho satura por volta de $m \approx 64$ – 128 — a árvore já é rasa demais. Existe um m ótimo**, do tamanho de um bloco de disco." "O mesmo aparece na altura."**

23. Impacto de m: altura. Mensagem: altura $\approx$ acessos. "A altura $\approx$ **acessos por busca**. Cai em **degraus**: $m=3 \rightarrow 14$ níveis (fundo demais); m grande $\rightarrow$ altura 2. Altura baixa é a razão de existir da Árvore B." "E nas três operações?"

24. Resultados por operação. Mensagem: o ganho vale para todas. "**Busca** é a mais barata (raiz $\rightarrow$ folha); **inserção** um pouco mais (split reescreve nós); **remoção** a mais cara (redistribui/funde). As três **caem com m e saturam juntas**." "E quando o conjunto cresce?"

25. Escala do conjunto. Mensagem: escalável. "De **mil a um milhão** de chaves, os acessos por busca **quase não mudam** — é custo **logarítmico na base m**. Escalável: cresce o dado, **não o custo**." "Para deixar esse ganho concreto, comparamos com um método mais simples."

****26. Comparação: Árvore B x Array ordenado. (*) Mensagem: por que a Árvore B existe. "Implementamos um baseline ingênuo — um array ordenado em disco, com o mesmo contador de acessos, para uma comparação justa. Resultado: na busca quase empatam, porque as duas são $O(\log n)$ (busca binária). Mas inserir e remover no array exige deslocar $\sim$ metade do arquivo ($O(n)$): com 10 mil chaves, são ~ 5 mil acessos por inserção contra ~ 9 da Árvore B — até $538\times$ mais I/O (e $\sim 440\times$ na remoção). A lição: ordenar é fácil; manter ordenado sob escrita é o que mata — e é isso que a Árvore B resolve com split/merge local**." "O segundo eixo da avaliação: espaço."**

27. Reaproveitamento de nós (churn). Mensagem: ~ 27 – 30% de economia. "No **churn** (insere, remove metade, reinsere), **com reuso** o arquivo praticamente **não cresce**; **sem reuso**, ele **incha**. A free list economiza ~ 27 – 30% do tamanho do arquivo neste workload." "Sobre o processo: validamos em duas máquinas."

28. Dois sistemas: tempo (wall). Mensagem: a forma é igual. "Mesmo programa, duas máquinas: os **acessos a disco são idênticos**; só o **tempo** muda. As curvas têm **a mesma forma**." "E por que o relógio engana?"

****29. Dois sistemas: CPU e determinismo. (*) Mensagem: idêntico nas duas. "À esquerda: o tempo é quase todo CPU de sistema (o I/O vai para o cache de páginas), por isso o relógio não é a métrica honesta. À direita: todos os pontos caem na reta $y=x$ — acessos rigorosamente iguais nas duas máquinas. Determinístico e portátil**."**

"Resumindo a validação."

30. Dois sistemas: validação cruzada. (~) Mensagem: portabilidade. "Notebook x Titan: acessos **idênticos**, só o tempo difere; comparação **automatizada** por `compare.py`."

"E a escala, nas duas?"

31. Dois sistemas: escala. Mensagem: tendência igual. "De mil a um milhão ($m=100$) nas duas máquinas: **mesma tendência**, Titan mais rápido em paralelo. **Prova prática de escalabilidade.**" "Um ponto que o professor levantou: o union."

****32.** E se usássemos union no header? (*) **Mensagem: página única x portabilidade.** "**Hoje header e BNode são** tipos separados, **com serialização** campo a campo (**memcpy**). **Com union, seriam uma** página única **de tamanho fixo: ler/gravar de uma vez**, menos código e menos bugs, **free list explícita**. **Custo:** portabilidade** (padding/endianness) e o UB de type-punning — resolvível com `memcpy/std::bit_cast`. Mais limpo, exigindo cuidado com o formato binário." "Quais foram as dificuldades?"

33. Dificuldades técnicas. Mensagem: o que custou. "Indexação **1-based** (RRN 0 = header); na **remoção**, manter o **path** para reparar underflow; o `eofbit` do `fstream` que silenciava leituras (resolvido com `clear()`); e a serialização campo a campo, verbosa e propensa a erro — foi ela que motivou a ideia do union." "E os trade-offs gerais?"

34. Vantagens x desvantagens. Mensagem: equilíbrio consciente. "Vantagens: **altura baixa** → poucos acessos, **sempre balanceada**, ideal para disco/índices, determinística com reuso de espaço. Custos: **nós subocupados** (~50% no sequencial), **remoção complexa**, e **m muito grande** troca I/O por CPU — daí o **m ótimo** (tamanho de bloco)." "Onde isso é usado de verdade?"

35. Aplicações práticas. Mensagem: está em tudo. "**SGBDs** (MySQL/InnoDB, PostgreSQL, Oracle), **sistemas de arquivos** (NTFS, APFS, ext4, Btrfs), **chave-valor** (SQLite, LMDB, BerkeleyDB). O uso natural é como **índice de um banco em disco**."

"Para fechar."

36. Conclusão. Mensagem: os achados. "Um: **Árvore B 100% em disco**, 1 nó por I/O. Dois: o **I/O por operação cai com m até saturar** (~64-128) — existe **m ótimo**. Três: a

free list economiza ~27-30% de espaço. Quatro: é **determinística** — acessos idênticos em duas máquinas. Rasa, balanceada e econômica: por isso é a base de bancos e sistemas de arquivos." "Referências e obrigado."

37. Referências. Comer (1979); Knuth, TAOCP vol. 3; Folk & Zoellick; Bayer & McCreight (1972); slides do Prof. Baranauskas. "Obrigado! Ficamos à disposição."

Perguntas prováveis (preparação)

"**Árvore B não é binária?**" → "Não. Binária é a BST (2 filhos). A Árvore B é

m-way (até m filhos); o 'B' é de balanceada (Bayer). m grande deixa a árvore rasa. Ordem 2 é degenerada e o código a proíbe; ordem 3 é só o menor caso testado."

"**Por que comparar com um array ordenado, e não com uma BST ou hash?**" → "Porque o array isola **exatamente** o ponto da Árvore B: as duas têm **busca $O(\log n)$** , então a busca quase empata; o que muda é a **escrita** — o array é $O(n)$ por deslocamento. Fica nítido que o ganho da Árvore B está em **manter a ordem barata sob inserção/remoção**. Usamos o **mesmo contador de acessos** nos dois, então é justo."

"**Por que o tempo aparece como CPU, não como espera de I/O?**" → "Porque `writeNode` faz `flush` para o **cache de páginas do SO**, não `fsync` no disco físico. O 'I/O' vira CPU de sistema. Por isso a métrica honesta é o **contador de acessos**."

"O que mudaria de fato com o union no header?" → "Header e nó virariam **uma página única** de tamanho fixo garantido em compilação: ler/gravar de uma vez, sem memcpy campo a campo, e free list explícita. Custo: portabilidade (padding/endianness) e UB de type-punning, resolvível com memcpy/std::bit_cast."

"Onde fica a raiz?" → "No header, registro 0 — uma leitura de metadado e já sabemos a raiz."

"Como funciona o reaproveitamento?" → "Lista de livres encadeada no arquivo: cabeça em free_head, cada livre guarda o próximo em K[1] com n=-1; allocNode puxa dela antes de estender o arquivo."

"Qual o m ideal na prática?" → "Aquele em que o nó preenche um **bloco de disco** (4-8 KB) — é onde a curva do slide 22 satura."

"Por que a inserção sequencial gera arquivo maior?" → "O split sempre cai à direita, deixando nós ~50% cheios; no aleatório a ocupação fica ~69% ($\ln 2$)."

Dicas de cronometragem

- **Alvo 45 min:** ~1:00-1:15 por slide. A tabela acima soma ~45.
- **Se atrasar:** encurte os marcados (~) (2, 11, 16, 30) lendo só o título e o destaque.
- **Se sobrar:** detalhe os marcados (*) (3, 4, 6, 22, 26, 29, 32) e abra perguntas no fim de cada parte.
- **Não leia os slides palavra por palavra** — a fala acima é a narração; os slides são o apoio visual.